

Arquitetura Detalhada do Aplicativo

Este documento descreve minuciosamente a arquitetura do aplicativo, abrangendo frontend, backend, lógica de acesso e conexões.

1. Tecnologias e Linguagens

- Frontend: React 18 com TypeScript (TSX).
- Build Tool: Vite para carregamento rápido e HMR.
- Estilização: Tailwind CSS para design responsivo e moderno.
- Backend: Supabase (PostgreSQL) como BaaS (Backend as a Service).
- Gerenciamento de Estado: Zustand para estado global e React Context para autenticação.
- Comunicação de Dados: TanStack Query (React Query) para sincronização e cache de dados.
- Ícones: Lucide React.
- Animações: Framer Motion e Tailwind Animate.

2. Arquitetura Frontend

O frontend é organizado de forma modular para facilitar a manutenção e escalabilidade.

- src/components: Contém componentes reutilizáveis, organizados por domínio (caixa, clientes, comercial, etc.).
- src/pages: Define as rotas do aplicativo, separando contextos Pessoal, PJ (Comercial) e Portais.
- src/hooks: Hooks customizados para lógica compartilhada e abstração de chamadas ao Supabase.
- src/contexts: Contextos React como AuthContext para gerenciar a sessão do usuário.
- src/stores: Zustand stores como useAppStore para estado volátil da aplicação.

3. Lógica de Login e Acesso

O sistema de autenticação é robusto e baseado em Supabase Auth.

- Auth.tsx: Gerencia o login via email/senha e solicitações de recuperação de senha.
- ProtectedRoute.tsx: Intercepta rotas protegidas, verifica o JWT e redireciona usuários não autenticados.
- Context Switcher: Permite ao usuário alternar entre o modo 'Pessoal' e 'Empresarial' (PJ).
- Níveis de Acesso: Admin (acesso total), Colaborador (painel simplificado), Cliente (portal de briefing/entrega), Super Admin (gestão de workspaces).

4. Arquitetura Backend (Supabase)

O banco de dados PostgreSQL é o coração do sistema, com segurança aplicada via Row Level Security (RLS).

- Tabelas Principais: projects, briefings, clients, tasks, finances (incomes/expenses), ai_agents.
- Row Level Security (RLS): Garante que usuários vejam apenas dados de seu próprio workspace ou atribuídos a eles.
- Edge Functions (Deno): Processamento server-side para tarefas pesadas como IA (ai-360-agent), convites de usuários e geração de links mágicos.
- Triggers e RPCs: Automações no banco de dados para manter a integridade e facilitar chamadas complexas do frontend.

5. Módulos e Funcionalidades (Botões e Ações)

Cada módulo possui botões de ação consistentes.

- Módulo Comercial: Botão 'Novo Cliente' (abre formulário de cadastro), 'Gerar Link' (cria token de acesso externo).
- Módulo de Projetos: Botão 'Criar Projeto', 'Adicionar Tarefa', 'Anexar Arquivo'.
- Módulo Financeiro: Botão 'Nova Receita/Despesa', 'Filtros de Data', 'Exportar Relatório'.
- Módulo IA: Botões de 'Chat', 'Gerar Resumo', 'Configurar Agente'.

6. Conexões e Sincronização

O aplicativo utiliza Realtime do Supabase para manter os dados sincronizados em múltiplos dispositivos.

- Sincronização em Tempo Real: Alterações em tarefas ou chats são refletidas instantaneamente via broadcasts e postgres_changes.
- Cache: TanStack Query mantém o estado local atualizado sem necessidade de refresh constante.
- Integrações de IA: Conexão direta com APIs da OpenAI/Anthropic via Edge Functions para assistentes inteligentes.

8. Esquema de Banco de Dados (Tabelas Principais)

O sistema utiliza as seguintes tabelas no esquema public:

- projects: Gestão de projetos de clientes e internos.
- briefings: Armazenamento de informações coletadas via formulários.
- clients: Cadastro de clientes e informações comerciais.

- `project_tasks`: Tarefas vinculadas a projetos específicos.
- `incomes / expenses`: Lançamentos financeiros para fluxo de caixa.
- `user_roles`: Definição de permissões por usuário.
- `workspaces`: Separação lógica de ambientes multi-tenant.
- `ai_agents`: Configurações de agentes de inteligência artificial customizados.
- `portal_short_links`: Redirecionamento de links curtos para acesso externo.
- `corporate_team`: Gestão de colaboradores e equipe.

9. Detalhamento de Botões e Funções

Abaixo, listamos botões críticos e sua lógica subjacente:

- Botão 'Alternar Contexto' (`ContextSwitcher.tsx`): Altera o estado 'mode' (personal/business) no store `useAppStore`, filtrando a visibilidade de menus e dados.
- Botão 'Salvar Briefing' (`BriefingEditorPage.tsx`): Executa uma mutação via React Query que envia os dados para a tabela 'briefings' e 'briefing_responses'.
- Botão 'Configurações' (`Sidebar.tsx`): Redireciona para `/config`, onde o usuário pode alterar preferências armazenadas na tabela 'user_preferences'.
- Botão 'Chat IA' (`AI360DashboardPage.tsx`): Abre uma conversa com o Edge Function 'ai-360-agent', enviando o histórico para processamento de linguagem natural.